L'API che stiamo usando per la programmazione delle socket in C fanno parte della specifica POSIX[1]. Questa API viene anche detta Berkeley sockets o BSD sockets, in omaggio alla sua prima implementazione nel sistema operativo BSD (Berkeley Software Distribution).

Il sistema operativo BSD, basato su Research UNIX, non è più sviluppato, ma oggi il termine viene comunemente associato a vari discendenti open source.

L'API per le socket può operare in due modalità:

- bloccante
- non bloccante

Noi stiamo usando l'API in modalità bloccante, che è la modalità predefinita.

In generale, invocate in modalità bloccante, le *system call* per le socket ritornano solo dopo il completamento dell'operazione, il verificarsi di un errore oppure l'interruzione della system call per la cattura di un segnale (il classico valore di ritorno *EINTR*). Quest'ultimo caso, in realtà, non lo tratteremo perché da anni *glibc* riavvia automaticamente molte system call.

---

socket(int *socket_family*, int *socket_type*, int *protocol*)

Crea una nuova socket e ne restituisce il file descriptor.

Parametri:

socket_family

famiglia di socket: associata a specifica famiglia di indirizzi. Per le socket Internet, si usano AF_INET (IPv4) e AF_INET6 (IPv6).

socket_type

tipo di socket: ovvero, la semantica della comunicazione.

- SOCK_STREAM: comunicazione orienta alla connessione, basata sul trasferimento affidabile, bidirezionale di un flusso di byte.
- SOCK_DGRAM: comunicazione senza connessione, basata sull'invio inaffidabile di pacchetti di dimensioni massima prestabilita

protocol:

Lo specifico protocollo da usare — per esempio IPPROTO_TCP per TCP e

IPPROTO_UDP per UDP — oppure lasciato al sistema, che lo deduce

[1] https://pubs.opengroup.org/onlinepubs/9699919799/functions/V2_chap02.html#tag_15_10

automaticamente se impostato a zero.

Valore restituito:

File descriptor della socket appena creata, oppure -1 in caso di errore (dettagliato nella variabile `errno` e stampabile con `perror`).

Note:

Appena creata, una socket non ha ancora un indirizzo locale associato.

- TCP: il client ottiene un indirizzo solo dopo `connect()`, mentre il server deve invocare `bind()` per assegnarne uno esplicitamente.
- UDP: una socket appena creata non può ricevere finché non viene fatto `bind()`. Può però inviare: al primo invio, il sistema assegna automaticamente un indirizzo locale alla socket.

L'indirizzo locale assegnato automaticamente viene scelto in base alla route utilizzata per raggiungere il peer, quindi corrisponde all'indirizzo IP dell'interfaccia di uscita.

`struct` **`sockaddr`**

Usato come tipo nelle API che gestiscono indirizzi di socket in generale. Ogni famiglia di socket utilizza una struttura specifica per la propria famiglia di indirizzi. Le socket AF_INET usano la struttura `sockaddr_in` (`sockaddr_in6` per AF_INET6), che contiene i campi `sin_family` (famiglia di indirizzi), `sin_addr` (indirizzo di rete) e `sin_port` (numero di porta); per IPv6 i campi corrispondenti hanno prefisso `sin6_`.

`INADDR_LOOPBACK`: 127.0.0.1 (è l'indirizzo della interfaccia di loopback)

`INADDR_ANY`: 0.0.0.0 (può essere usato nella bind per associare la socket a tutte le interfacce locali)

Nota: il valore di queste due costanti è in realtà il valore numerico degli indirizzi IP indicati.

---

struct **sockaddr_storage**

Struttura usata per definire variabili associate a un'area di memoria sufficientemente ampia e correttamente allineata, al fine di contenere qualsiasi struttura di indirizzo di socket.

`int` **`bind`**`(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`

Assegna alla socket un indirizzo locale.

Parametri:

sockfd

file descriptor della socket

addr

puntatore alla struttura che rappresenta l'indirizzo da assegnare alla socket

addrlen

lunghezza effettiva della struttura dell'indirizzo

Valore restituito:

0 in caso di successo, -1 in caso di errore (con il codice dettagliato in `errno`).

Note:

Non è necessario per il client; vedi la spiegazione relativa a `socket()`.

`int` **`listen`**`(int sockfd, int backlog)`

Rende la socket indicata passiva, cioè utilizzabile per accettare richieste di connessione.

Parametri:

sockfd

file descriptor della socket

backlog

numero di connessioni completamente stabilite che possono essere accodate in attesa di essere accettate dal server. Il valore viene troncato se supera il limite massimo di sistema, consultabile in `/proc/sys/net/core/somaxconn`

Valore restituito:

0 in caso di successo; -1 in caso di errore (con il codice riportato in `errno`).

`int` **`accept`**`(int sockfd, struct sockaddr *_Nullable restrict addr, socklen_t *_Nullable restrict addrlen)`

Preleva una connessione completamente stabilita dal backlog di una socket orientata alle connessioni, crea una nuova socket connessa per tale connessione e restituisce il file descriptor che la rappresenta. La chiamata rimane bloccata finché non è disponibile una connessione da prelevare.

Parametri:

sockfd

file descriptor della socket in ascolto

addr

puntatore a un'area di memoria in cui verrà scritto l'indirizzo della socket peer

addrlen

puntatore a una variabile di tipo `socklen_t`; deve contenere inizialmente la dimensione del buffer puntato da `addr` e viene poi aggiornata con la lunghezza effettiva dell'indirizzo, in funzione della address family. Se il buffer è troppo piccolo, l'indirizzo viene troncato e il nuovo valore di `addrlen` è maggiore di quello iniziale.

Valore restituito:

file descriptor della nuova socket connessa, oppure -1 in caso di errore.

Note:

gli ultimi due parametri possono essere impostati a zero se non si desidera ottenere informazioni sull'indirizzo del peer.

`int` **`getsockname`**`(int sockfd, struct sockaddr *restrict addr, socklen_t *restrict addrlen)`

Restituisce l'indirizzo cui è associata una socket.

Parametri:

sockfd

file descriptor della socket

addr

puntatore a un buffer nel quale scrivere l'indirizzo

addrlen

puntatore a una variabile di tipo `socklen_t`; deve contenere inizialmente la dimensione del buffer puntato da `addr` e viene poi aggiornata con la lunghezza effettiva dell'indirizzo, in funzione della address family. Se il buffer è troppo piccolo, l'indirizzo viene troncato e il nuovo valore di `addrlen` è maggiore di quello iniziale.

Valore restituito:

0 in caso di successo; -1 in caso di errore (con il codice riportato in `errno`).

Note:

Gli ultimi due parametri si comportano come gli analoghi di `accept()`.

`int` **`getpeername`**`(int sockfd, struct sockaddr *restrict addr, socklen_t *restrict addrlen)`

Parametri:

sockfd

file descriptor della socket

addr

puntatore a un buffer nel quale scrivere l'indirizzo

addrlen

puntatore a una variabile di tipo `socklen_t`; deve contenere inizialmente la dimensione del buffer puntato da `addr` e viene poi aggiornata con la lunghezza effettiva dell'indirizzo, in funzione della address family. Se il buffer è troppo piccolo, l'indirizzo viene troncato e il nuovo valore di `addrlen` è maggiore di quello iniziale.

Valore restituito:

0 in caso di successo; -1 in caso di errore (con il codice riportato in `errno`).

Note:

Gli ultimi due parametri si comportano come gli analoghi di `accept()`.

`read`, **`recv`**, **`recvfrom`**:

Servono per leggere da una socket. Si bloccano finché il buffer di ricezione è vuoto. Restituiscono il numero di byte letti, che può essere inferiore a quanto richiesto; restituiscono -1 in caso di errore.

`recv` e recvfrom hanno un parametro aggiuntivo per specificare opzioni.

`recvfrom` si usa con socket non connesse per ottenere l'indirizzo del mittente, con la stessa logica dei due parametri finali di `accept`().

`write`, **`send`**, **`sendto`**

Servono per scrivere su una socket. Si bloccano finché il buffer di invio è pieno. Restituiscono il numero di byte effettivamente scritti, che può essere inferiore a quanto richiesto, oppure -1 in caso di errore. È importante notare che i byte passati a queste chiamate di sistema non sono necessariamente già stati trasmessi sul collegamento: possono trovarsi ancora nei buffer del kernel.

Poiché queste funzioni non attendono l'effettiva trasmissione dei dati sul collegamento, le chiamate `write`, `send` e `sendto` possono restituire errore solo per condizioni "locali" (socket non valida, permessi, ecc.). Eventuali errori di comunicazione potrebbero essere segnalati solo in corrispondenza di chiamate successive, a causa di un cambiamento dello stato della socket.

TCP vs UDP:

- TCP tratta il flusso come una sequenza continua di byte: i dati presenti nel buffer di ricezione e non ancora letti tramite `read`/`recv` possono essere letti successivamente. È responsabilità dell'applicazione riconoscere i confini dei messaggi (framing). Alcune tecniche comuni sono: dimensione fissa, campo lunghezza, delimitatore, oppure E0F (quest'ultimo poco affidabile, perché la connessione potrebbe essere stata chiusa per errore).
  Nota: l'opzione MSG_WAITALL usato in un recv su socket TCP permette di bloccare la chiamata finché non sono stati letti tutti i byte, la connessione è stata chiusa, oppure si è verificato un errore. La chiusura della connessione oppure lo scadere del timeout di lettera (impostato tramite opzione) possono dare luogo a letture parziali.
  Una lettura può restituire 0 solo per segnalare la condizione di EOF, oppure se è stato chiesto di leggere zero byte.
- UDP tratta i dati come datagrammi discreti: il contenuto di un datagramma deve essere letto con una singola operazione. Se il buffer fornito è troppo piccolo, i byte eccedenti vengono scartati. In maniera duale, un singolo messaggio applicativo deve essere trasmesso con una sola scrittura: la sua dimensione è inoltre limitata—pena errore—dalla dimensione massima del datagramma UDP (dovuta alla presenza di un campo di

16 bit per la lunghezza complessiva del datagramma); quest'ultimo limite spesso ulteriormente limitato dal valore corrente della path MTU, quando il sistema è configurato per evitare la frammentazione del datagramma IP nell'host stesso di invio. Una lettura può restituire 0 se il datagramma UDP contiene zero byte di dati (oppure è stato chiesto di leggere zero byte).

`ntohl`, **`ntohs`**

Conversione da big endian (***n****etwork byte order*) all'endian locale (***h****ost byte order*) di interi a 32 e 16 bit, rispettivamente.

`htonl`, **`htnos`**

Conversione dall'endian locale (***h****ost byte order*) a big endian (***n****etwork byte order*) di interi a 32 e 16 bit, rispettivamente.

Note:

Endian indica l'ordine in cui i byte di un valore multibyte (come un intero a 16 o 32 bit) vengono memorizzati in memoria.

- **Big endian**: il byte più significativo viene memorizzato per primo.
- **Little endian**: il byte meno significativo viene memorizzato per primo.

Le API delle socket utilizzano il network byte order (big endian) per i numeri di porta e gli indirizzi di rete.

`int` **`inet_pton`**`(int af, const char *restrict src, void *restrict dst)`

Converte una stringa in un indirizzo di rete appropriato per la address family specificata.

Parametri:

af

famiglia di indirizzi, ad esempio AF_INET o AF_INET6

src

stringa da convertire

dst

puntatore a un buffer in cui verrà memorizzata la struttura per un indirizzo di rete

Valore restituito:

1 in caso di successo**,** 0 se *src* non rappresenta un indirizzo valido, oppure -1 se *af* non è una famiglia di indirizzi supportata.

`const char *`**`inet_ntop`**`(int af, const void *restrict src, char dst[restrict .size], socklen_t size)`

Converte una struttura che rappresenta un indirizzo di rete in una stringa.

af

famiglia di indirizzi, ad esempio AF_INET o AF_INET6

af

puntatore a una struttura per l'indirizzo di rete da convertire

dst

puntatore a un buffer nel quale verrà memorizzata la stringa che rappresenta l'indirizzo di rete

size

dimensione del buffer

Valore restituito:

il valore di *dst* in caso di successo; altrimenti *NULL* in caso di errore dettagliato in `errno`.

`int setsockopt(int sockfd, int level, int optname, const void * optval[.optlen], socklen_t optlen)`

Imposta un'opzione su una socket.

Parametri:

sockfd

file descriptor della socket

level

livello dell'opzione: SOL_SOCKET per opzioni di livello socket; altrimenti l'identificatore numerico del protocollo per opzioni specifiche (es. TCP, IP).

optname

identificatore dell'opzione da impostare

optval

puntatore al valore da assegnare (spesso un `int`)

optlen

lunghezza del buffer che contiene il valore dell'opzione

Valore restituito:

0 in caso di successo; -1 in caso di errore (dettagliato in `errno`).

Note:

SO_REUSEADDR permette il riutilizzo dell'indirizzo (indirizzo di rete e numero di porta) in quattro casi:

1. **TIME_WAIT** Il caso più comune: riavvio immediato del server.
2. **Bind su indirizzo specifico** Se un processo fa bind su 0.0.0.0:9000, un altro può fare bind su 192.168.1.10:9000.
3. **Multicast** Più processi possono unirsi allo stesso gruppo multicast.
4. **Broadcast** Permette di inviare pacchetti broadcast sulla stessa porta.

Se voglio impostare un timeout per le letture (SO_RCVTIMEO) e per le scritture (SO_SNDTIMEO).

```
struct timeval timeo;
timeo.tv_sec = 5;       // 5 secondi
timeo.tv_usec = 0;    // e zero microsecondi; meno di 1 milione

setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, &timeo, sizeof(timeo));
setsockopt(sockfd, SOL_SOCKET, SO_SNDTIMEO, &timeo, sizeof(timeo));
```

`int` **`connect`**`(int sockfd, const struct sockaddr *addr, socklen_t addrlen);`

Connette la socket all'indirizzo specificato in addr.

Parametri:

sockfd

file descriptor della socket

addr

puntatore a una struttura indirizzo di socket

addrlen

dimensione della struttura puntata

Valore restituito:

0 in caso di successo; -1 in caso di errore (con codice in errno).

Note:

- **TCP:** la chiamata ritorna solo al termine del 3-way handshake oppure al verificarsi di un errore.
- **UDP:** ritorna immediatamente, dopo aver impostato l'indirizzo remoto (destinazione predefinita e unica sorgente accettata).

---

int **getaddrinfo**(const char *restrict node,

const char *restrict service,

const struct addrinfo *restrict hints,

struct addrinfo **restrict res)

Converte stringhe leggibili (nomi di host come "google.com" o indirizzi IP) e servizi (come "http" o "80") in strutture di indirizzi di rete pronte all'uso.

Parametri:

node

Il nome dell'host (es. "www.google.com"), un indirizzo IP in formato stringa (es. "192.168.1.1" o "2001:db8::1"), oppure NULL. Se è NULL, il comportamento dipende dai flag impostati in hints (es. permette di fare il bind su tutte le interfacce se si usa AI_PASSIVE: altrimenti usa l'indirizzo della interfaccia di loopback).

service

Il numero di porta in formato stringa (es. "80"), il nome testuale del servizio (es. "http", risolto tramite il file /etc/services), oppure NULL.

hints

Puntatore a una struttura `addrinfo` utilizzata per specificare i criteri di filtro e configurazione (es. `ai_family` per forzare IPv4 (`AF_INET`) / IPv6 (`AF_INET6`), `ai_socktype` per `SOCK_STREAM` / `SOCK_DGRAM`, e `ai_protocol` per `IPPROTO_TCP`/`IPROTO_UDP`). Può essere NULL se si vogliono accettare i valori di default.

res

Puntatore a un puntatore (un parametro di output) che la funzione utilizzerà per restituire l'indirizzo della testa della lista concatenata di strutture `addrinfo` allocate dinamicamente.

Valore restituito:

0 in caso di successo; altrimenti un codice di errore (per il quale si può ottenere una rappresentazione testuale usando `gai_strerror`).

Note:

`getaddrinfo` alloca dinamicamente una lista concatenata di strutture `addrinfo` che, dopo l'uso, deve essere liberata utilizzando `freeaddrinfo`.

Almeno uno tra `node` e `service` deve essere non-NULL.

---

void **freeaddrinfo**(struct addrinfo *res)

const char ***gai_strerror**(int errcode)

```
char hoststr[NI_MAXHOST];
char portstr[NI_MAXSERV];
int s = getnameinfo((struct sockaddr *)&peeraddr, peeraddrlen,
                            hoststr, sizeof(hoststr),
```

```
                                portstr, sizeof(portstr),
                                NI_NUMERICHOST | NI_NUMERICSERV);
```

`getnameinfo` può essere usata per tradurre l'indirizzo di rete, contenuto in una struttura di indirizzo generica, nella rispettiva rappresentazione numerica sotto forma di stringa (utilizzando i flag specifici). In questo modo, si evita la necessità di verifica la famiglia (sa_family) e poi fare il cast al tipo appropriato per accedere ai vari field.

Restituisce 0 in caso di successo; altrimenti un codice di errore (per il quale si può ottenere una rappresentazione testuale usando `gai_strerror`).

---

int **shutdown**(int sockfd, int how)

Chiude tutto o in parte una connessione full-duplex.

Parametri:

sockfd

file descriptor della socket

how

SHUT_RD: disabilita ulteriori ricezioni

SHUT_WR: disabilita ulteriori trasmissioni → programma l'invio di un segmento FIN (la cui ricezione indica al peer che non riceverà ulteriori byte di dati, permettendogli di rilevare la condizione di EOF dopo che ha letto tutti dati ricevuti precedentemente)

SHUT_RD: disabilita ulteriori ricezioni e trasmissioni

Valore restituito:

0 in caso di successo; -1 in caso di errore (con il codice riportato in `errno`).

Note:

`shutdown(..)` non va confusa con `close(..)`:

- `shutdown(..)` opera sulla connessione, a prescindere dal numero di file descriptor per essa: inoltre, il file descriptor usato come argomento può essere usato successivamente
- `close(..)` chiude innanzitutto il file descriptor, decrementare il reference count sulla socket: solo quando quest'ultimo arriva a zero la socket viene chiusa. In ogni caso, il file descriptor usato come argomento non è poi più valido.

`shutdown(..)` e, in generale, `close(..)` ritornano immediatamente: non attendono l'invio del FIN né il completamente delle trasmissioni, etc.